

DBMS + SQL Interview Notes

Canonical answers for spaced repetition + practical SQL patterns

Roadmap scope DBMS Fundamentals: Database vs DBMS, relational database, keys, normalization, denormalization, transactions, ACID, constraints, indexes, joins. SQL practice: six roadmap queries.

How to Use These Notes

- Memorize the bold/core definition first; do not try to memorize every sentence word-for-word.
- For each topic, retrieve the sequence: Definition -> Why -> Example -> Key distinction/trade-off.
- For SQL, first understand the table relationship and expected rows, then write the query.
- During spaced repetition, cover the answer and explain from the question title alone.

Canonical-answer rule Use one stable definition as your anchor. When speaking, you can simplify the wording, but keep the technical meaning unchanged.

1. DBMS Fundamentals

1.1 Database vs DBMS

Definition A database is an organized collection of data. A DBMS (Database Management System) is software that lets users/applications create, store, retrieve, update, secure, and manage data in databases.

Why DBMS exists

- Provides controlled data access instead of making every application manage raw storage itself.
- Supports querying, transactions, concurrency, security, backup/recovery, and integrity mechanisms.

Example

- A PostgreSQL server is a DBMS; an application database containing users, orders, and products is the managed data collection.

Key distinction

Database	DBMS
The data itself / organized collection	Software that manages the data
Stores the information	Provides operations and controls for the information
Example: a set of application tables	Example: PostgreSQL, MySQL, Oracle Database

1.2 Relational Database

Definition A relational database stores data in tables (relations) made of rows and columns, and represents relationships between tables using keys and constraints.

Core ideas

- Row = one record/tuple; column = an attribute/field.
- Primary/unique constraints identify records; foreign keys connect related tables.

- SQL is commonly used to query relational databases.

Example

Table	Columns	Meaning
Customers	customer_id, name, email	One row per customer
Orders	order_id, customer_id, total	One row per order; customer_id references Customers
OrderItems	order_id, product_id, qty	Line items belonging to orders

1.3 Keys: Super, Candidate, Primary, Composite, Foreign

Super key

Definition A super key is any set of one or more attributes that uniquely identifies a row. It may contain extra attributes that are not necessary for uniqueness.

Candidate key

Definition A candidate key is a minimal super key: it uniquely identifies a row, and removing any attribute would destroy that uniqueness.

Primary key

Definition A primary key is the candidate key selected as the table's main identifier. It must uniquely identify each row and cannot be NULL.

Foreign key

Definition A foreign key is a column or set of columns whose values reference a candidate/primary key in another table, helping enforce referential integrity.

Composite key

Definition A composite key uses two or more columns together as one key. It is useful when no single column uniquely identifies a row.

Composite-key example

```
CREATE TABLE Enrollment (
  student_id INT,
  course_id INT,
  semester INT,
  PRIMARY KEY (student_id, course_id, semester)
);
```

Memory chain Super key = unique; Candidate key = minimal unique; Primary key = chosen candidate; Foreign key = reference to another table; Composite key = key made from multiple columns.

1.4 Normalization

Definition Normalization is the process of organizing relational data to reduce unnecessary redundancy and prevent update, insertion, and deletion anomalies.

Why normalize?

- Reduce duplicated facts.
- Keep each fact stored in an appropriate place.
- Improve consistency and maintainability.

The anomaly idea

Think in anomalies Update anomaly: the same fact must be changed in multiple rows. Insert anomaly: you cannot store a fact without unrelated data. Delete anomaly: deleting one fact accidentally removes another useful fact.

1.5 First Normal Form (1NF)

Definition A relation is in 1NF when each field contains a single atomic value and repeating groups or multi-valued cells are removed.

Bad -> better

Not 1NF	1NF form
Student: Phones = "9876, 8765"	Separate StudentPhone rows: (student, phone)
Order: Items = "Pen, Book, Bag"	Separate OrderItem rows

1.6 Second Normal Form (2NF)

Definition A relation is in 2NF when it is in 1NF and every non-key attribute is fully functionally dependent on the whole candidate key, not only part of a composite key.

Important

- Partial dependency matters only when the key is composite.
- A table with a single-column key cannot have a partial dependency on part of that key.

Example

Enrollment(student_id, course_id, student_name, course_name, grade)

Key = (student_id, course_id)

student_name depends only on student_id

course_name depends only on course_id

grade depends on the whole key

- To move toward 2NF, separate Student(student_id, student_name) and Course(course_id, course_name), leaving Enrollment(student_id, course_id, grade).

1.7 Third Normal Form (3NF)

Definition A relation is in 3NF when it is in 2NF and non-key attributes do not depend transitively on another non-key attribute.

Example

```
Employee(emp_id, emp_name, dept_id, dept_name)
```

```
emp_id -> dept_id
```

```
dept_id -> dept_name
```

```
therefore emp_id -> dept_name transitively
```

- Separate Department(dept_id, dept_name) from Employee(emp_id, emp_name, dept_id).

Memory chain 1NF = atomic values. 2NF = remove partial dependency. 3NF = remove transitive dependency.

1.8 Denormalization

Definition Denormalization deliberately introduces controlled redundancy to improve read performance or simplify read-heavy workloads, usually at the cost of extra storage and more complex writes.

Why use it?

- Reduce expensive joins for frequent reads.
- Precompute or duplicate data needed by performance-critical queries.

Trade-off

- More redundancy can cause inconsistency unless application/database logic keeps duplicated values synchronized.

1.9 Transactions

Definition A transaction is a logical unit of database work that should be completed as a consistent whole: it either commits successfully or is rolled back when it cannot complete as intended.

Example

```
BEGIN;
```

```
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
```

```
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
```

```
COMMIT;
```

- If a required step fails before COMMIT, the transaction can be rolled back so a partial transfer is not left behind.

1.10 ACID

Property	Meaning	Bank-transfer intuition
Atomicity	All-or-nothing transaction outcome.	Debit and credit both happen, or neither does.
Consistency	A committed transaction preserves database rules/invariants.	Balances and constraints remain valid after commit.
Isolation	Concurrent transactions should behave according to the database isolation guarantees.	One transfer should not observe an invalid half-finished state.
Durability	Committed changes survive failures according to the DBMS durability guarantees.	After commit, data should survive a crash/restart.

Recall Atomicity = all or nothing; Consistency = valid state; Isolation = concurrency separation; Durability = committed

changes persist.

1.11 Constraints

Definition Constraints are rules enforced by the database to protect data integrity and valid relationships.

Constraint	Purpose	Example
PRIMARY KEY	Unique row identity; not NULL.	customer_id
FOREIGN KEY	Referential integrity between tables.	Orders.customer_id -> Customers.customer_id
UNIQUE	Prevents duplicate values in a column/set.	email
NOT NULL	Requires a value.	name NOT NULL
CHECK	Restricts values to a condition.	age >= 0
DEFAULT	Supplies a value when one is omitted.	status DEFAULT 'active'

1.12 Indexes

Definition An index is an auxiliary data structure that helps the DBMS locate rows more efficiently for supported queries, reducing the need to scan the whole table in many cases.

Why indexes help

- Speed up selective reads such as WHERE lookups, joins, and some ORDER BY/GROUP BY operations when the optimizer can use the index.

Trade-offs

- Indexes consume additional storage.
- INSERT, UPDATE, and DELETE can become more expensive because relevant indexes must be maintained.
- Too many or poorly chosen indexes can hurt performance; the optimizer may choose a full scan when that is cheaper.

Conceptual example

```
CREATE INDEX idx_employee_email  
ON Employee(email);
```

Important An index is not automatically faster for every query. Selectivity, table size, query shape, statistics, and the optimizer all matter.

1.13 Joins

Definition A JOIN combines rows from multiple tables according to a matching condition, usually a relationship between keys.

Join	What it keeps	Typical use
INNER JOIN	Only rows with matches on both sides.	Customers who have orders.
LEFT JOIN	All rows from left + matching right rows; unmatched right values are NULL.	All customers, including those with no orders.

Join	What it keeps	Typical use
RIGHT JOIN	All rows from right + matching left rows.	Less common when the query can be rewritten with LEFT JOIN.
FULL OUTER JOIN	All rows from both sides; unmatched side is NULL.	Compare two sets while retaining unmatched rows.
CROSS JOIN	Cartesian product of both inputs.	Generate combinations intentionally.

INNER JOIN example

```
SELECT c.name, o.order_id
FROM Customers c
INNER JOIN Orders o
  ON o.customer_id = c.customer_id;
```

LEFT JOIN example

```
SELECT c.name, o.order_id
FROM Customers c
LEFT JOIN Orders o
  ON o.customer_id = c.customer_id;
```

Most important distinction INNER JOIN answers "who matches?" LEFT JOIN answers "keep everyone from the left, whether matched or not."

2. SQL Practice - Roadmap Problems

Exam method Before writing SQL: identify the requested rows, required tables, join keys, grouping level, and whether the question asks for a row-level filter, aggregate, or ranking.

2.1 Second Highest Salary

Problem Return the second highest distinct salary. If a second distinct salary does not exist, the result should be NULL.

Canonical solution - DENSE_RANK

```
SELECT salary
FROM (
  SELECT salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM Employee
) x
WHERE rnk = 2;
```

Alternative - MAX below the maximum

```
SELECT MAX(salary) AS second_highest_salary
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Why DENSE_RANK handles duplicate salaries correctly because equal salaries share the same rank. The MAX-below-MAX version also returns the second distinct value.

2.2 Employees Earning More Than Their Managers

Problem Return employees whose salary is greater than the salary of their manager.

```
SELECT e.name AS employee
FROM Employee e
JOIN Employee m
  ON e.managerId = m.id
WHERE e.salary > m.salary;
```

Pattern Self join: the same table plays two roles. e = employee row; m = manager row.

2.3 Duplicate Emails

Problem Return email addresses that occur more than once in the Person table.

```
SELECT email
FROM Person
GROUP BY email
HAVING COUNT(*) > 1;
```

Pattern GROUP BY creates one group per email; HAVING filters groups after aggregation.

2.4 Department Highest Salary

Problem For each department, return the employee(s) with the highest salary in that department.

Robust solution - DENSE_RANK per department

```
SELECT department_name, employee_name, salary
FROM (
  SELECT d.name AS department_name,
         e.name AS employee_name,
         e.salary,
         DENSE_RANK() OVER (
           PARTITION BY e.departmentId
           ORDER BY e.salary DESC
         ) AS rnk
  FROM Employee e
  JOIN Department d
    ON e.departmentId = d.id
) x
WHERE rnk = 1;
```

Why DENSE_RANK If two employees tie for the highest salary, both are returned. ROW_NUMBER would keep only one row.

2.5 Customers Who Never Order

Problem Return customers for whom no matching order exists.

```
SELECT c.name
FROM Customers c
LEFT JOIN Orders o
  ON o.customerId = c.id
WHERE o.id IS NULL;
```

Alternative - NOT EXISTS

```
SELECT c.name
FROM Customers c
WHERE NOT EXISTS (
  SELECT 1
  FROM Orders o
  WHERE o.customerId = c.id
);
```

Pattern LEFT JOIN + IS NULL and NOT EXISTS are classic anti-join patterns. Be careful not to put the order filter in WHERE when you need to preserve unmatched customers.

2.6 Customers With More Than One Order

Problem Return customers who have placed at least two orders.

```
SELECT c.id, c.name
FROM Customers c
JOIN Orders o
  ON o.customerId = c.id
GROUP BY c.id, c.name
HAVING COUNT(o.id) > 1;
```

Pattern JOIN creates the customer-order rows; GROUP BY moves the query to one row per customer; HAVING filters customers by aggregate count.

3. SQL Patterns You Should Recognize

Question wording	Pattern to think about	Typical SQL
"more than K" / "at least K"	Group then filter aggregate	GROUP BY + HAVING
"duplicate values"	Count each value/group	GROUP BY + COUNT
"highest per department"	Rank within each group	DENSE_RANK / ROW_NUMBER + PARTITION BY
"never / no matching row"	Anti-join	LEFT JOIN + IS NULL / NOT EXISTS
"employee vs manager"	Same table in two roles	Self JOIN
"second highest"	Distinct ordering / rank	DENSE_RANK or MAX below MAX

4. DBMS Rapid Recall

Database vs DBMS? Database = organized data; DBMS = software that manages it.

What is a relational database? Tables + rows/columns + relationships represented with keys/constraints.

Super key vs candidate key? Candidate key is a minimal super key.

Candidate key vs primary key? Primary key is the chosen candidate key.

What is a foreign key? A column/set of columns referencing a key in another table.

What is normalization? Organizing data to reduce redundancy and anomalies.

1NF? Atomic values; no repeating groups.

2NF? 1NF + no partial dependency on part of a composite key.

3NF? 2NF + no transitive dependency of non-key attributes.

Why denormalize? Controlled redundancy for read performance/simpler reads.

What is a transaction? A logical unit of work that commits as a whole or rolls back.

ACID? Atomicity, Consistency, Isolation, Durability.

Why constraints? Protect data integrity and valid relationships.

Why indexes? Speed supported reads, at cost of storage and write maintenance.

INNER vs LEFT JOIN? INNER keeps matches; LEFT keeps all left rows plus matches.

5. Spaced-Repetition Checklist

Review method Day 0: learn. Day 1: recall. Day 3: recall. Day 7: recall. Day 14: recall. For SQL, rewrite each query from memory and test it with a tiny handmade dataset.

Review target	You should be able to recall
Keys	Super -> Candidate -> Primary; Foreign; Composite
Normalization	1NF -> 2NF -> 3NF and the anomaly/dependency each addresses
ACID	All four names + one-line meaning
Indexes	Purpose + write/storage trade-off
Joins	INNER, LEFT, RIGHT, FULL OUTER, CROSS
SQL aggregation	GROUP BY vs HAVING
SQL ranking	DENSE_RANK, ROW_NUMBER, PARTITION BY
SQL anti-join	LEFT JOIN + IS NULL / NOT EXISTS
Self join	Same table in two aliases for different roles

Roadmap alignment: This note set follows the DBMS Fundamentals + SQL Practice scope in the provided Infosys preparation roadmap.